**CSCE 411: Design and Analysis of Algorithms (Honors)**

# Week 10: Computational Complexity: P and NP

*Date: October 27, 2026* *Nate Veldt, updated by Samson Zhou*

## 1 Reductions

A reduction is a mapping from one computational problem to another.

**Definition** Let $A$ and $B$ be decision problems. We say that $A$ can be reduced to $B$ if

- For every instance of $A$ we can define a corresponding instance of $B$ such that
- All YES instances in $A$ map to YES instances in $B$
- All NO instances in $A$ map to NO instances in $B$

This is a polynomial-time reduction if this conversion process takes polynomial time.

This seems obvious: what's an example of a reduction that is not polynomial-time?

## 1.1 Reduction and complexity

If $A$ can be reduced to $B$ in polynomial time, we denote this by

_____

This implies that:

- $A$ is easier[1] than $B$, or equivalently
- $B$ is at least as hard as $A$ is to solve

**Lemma 1.1.** *If* $B \in P$ *and* _____, *then* _____

[1] where easier might mean "no harder than"

## 2 NP-completeness

**Definition:** A problem $Q$ is NP-complete if:

1. $Q \in$ NP
2. for every problem $B \in$ NP, $B \leq_p Q$.

**In words:**

This implies that NP-complete problems are the *hardest* problems in NP.

Why? Remember that $A \leq_p B$ means $A$ is easier than $B$.

In fact, if you only take the second part of the definition, this defines the set of NP-hard problems.

## 3 Views of the computational problem universe

There are four different possibilities for the relationship between P, NP, and co-NP.

There are two possibilities for the relationship between P, NP, and NP-complete.

**A few famous NP-complete problems**

- Find if a graph has a clique of size at least $k$
- Find if a graph has a Hamiltonian cycle
- Find if a graph has a simple path with at least $k$ edges
- **Subset sum**: given a set of numbers, find if any subset sums to zero
- **SAT**: Logic problems we will look at in more depth later

## 4 Summary on Reduction and NP-completeness

If $A$ can be reduced in polynomial time to $B$, we write $A \leq_p B$. This means that $A$ is no harder than B; a solver for $B$ can be used to solve $A$.

A problem $Q$ is NP-hard if every problem in NP can be reduced to $Q$ in polynomial time.

A problem $Q$ is NP-complete if

- It is in NP
- It is NP-hard

**Question 1.** *Assume that $A$ and $B$ are both in NP and $A \leq_p B$. Which of the following statements is false?*

A *If $A \in P$, then $B \in P$*

B *If $A$ is NP-complete, then $B$ is NP-complete*

C *If $B \in P$, then $A \in P$*

D *If $A$ is NP-complete, then $B \leq_p A$*

If two problems are NP-complete, then they are both in NP, and:

## 5 Why does it matter whether a problem is in P, NP, or is NP-complete?

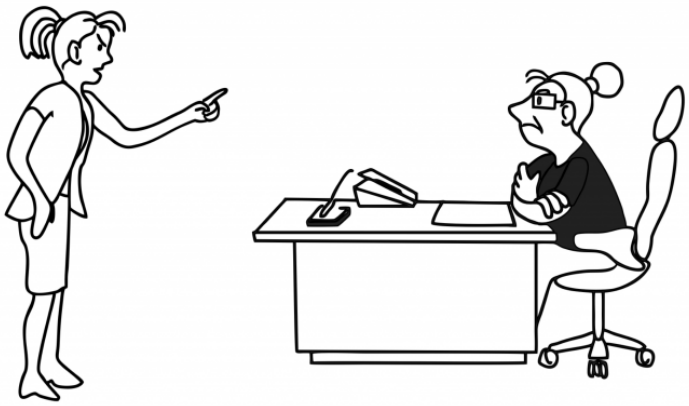
A famous book by Garey and Johnson, *Computers and Intractability*, includes some famous cartoons to help us answer this question.[2]

Let's say your boss tells you that because you are a CS major or minor you should be able to find an efficient algorithm for a certain problem at work. You work for weeks without progress. You don't want to tell your boss:



**"I can't find an efficient algorithm, I guess I'm just too dumb."**

Ideally you'd like to be able to say:



**"I can't find an efficient algorithm, because no such algorithm is possible!"**

But in most situations, proving definitively that this is impossible is unlikely.

[2]Garey, Michael R., and David S. Johnson. Computers and intractability. Vol. 174. San Francisco: freeman, 1979. High-resolution versions of the cartoon are available online.

If you fail to find an efficient solution to a problem, ask yourself: am I struggling to find a solution simply because this is NP-complete? If you can prove it is, you have a great answer for your boss.



**"I can't find an efficient algorithm, but neither can all these famous people."**

## 6 A brief and incomplete history of NP-completeness

- 1971: Stephen Cook proves that every problem in NP can be reduced to a problem called *Boolean Satisfiability* (SAT).

  In modern terminology: SAT is NP-complete.

  The same results were developed independently by someone named Leonid Levin, so this is now called the *Cook-Levin* Theorem.

- 1972: Richard Karp publishes a paper *Reducibility Among Combinatorial Problems*, showing that SAT can be reduced to 21 other problems.

- 1979: Garey and Johnson publish the go-to reference on NP-completeness: *Computers and Intractability.*

## 7 Boolean Satisfiability Problems

A *boolean variable* is a variable $x$ that can take on the values True or False.

A *boolean expression* or *boolean formula* is built from

- Variables: $x_1, x_2, \ldots, x_n$, all in $\{\text{True}, \text{False}\}$
- Operators:
  - AND ($\wedge$): e.g., True $\wedge$ False = False
  - OR ($\vee$): e.g., True $\vee$ False = True
  - NOT: $\neg$True = False; $\neg$False = True

    This is often also denoted with an overline:
- Parentheses: telling you the order in which to evaluate statements

Here are some examples of Boolean expressions:

- $x_1 \wedge x_2 \wedge x_3$
- $\neg x_2 \vee x_3$
- $(\neg x_1 \wedge x_2) \vee (x_1 \wedge x_2 \wedge x_3) \vee (x_2 \vee x_3)$

**Question 2.** *Does the following statement evaluate to True or False?*

$$(T \wedge F) \vee (T \wedge F \wedge T) \vee (F \vee T)$$

A *True*

B *False*

**Question 3.** *Is the following Boolean expression satisfiable or not?*

$$(x_1 \wedge x_2) \vee (x_1 \wedge x_2 \wedge x_3) \vee (\neg x_1 \vee x_3)$$

A *Yes, it's satisfiable*

B *No, it's not satisfiable*

**Theorem 7.1.** *Checking whether a boolean formula is satisfiable is* _____

We will not prove this, but we'll use it as a starting point for proving other problems are NP-complete.

## 8 CNF Satisfiability and 3SAT

Some more terminology

- Literal: a boolean variable or its negation: e.g.,

- Clause: the OR of multiple literals: e.g.,

- A boolean formula is in conjunctive normal form if it is the AND of clauses.

A boolean formula is in 3-CNF if _____

**Theorem 8.1.** *Checking whether a boolean formula in 3-CNF is satisfiable is NP-complete. This is called 3SAT.*

Again: we'll use it as a starting point for proving other problems are NP-complete.

## 9 The clique problem is NP-complete

**The clique problem:** Check if an unweighted and undirected graph $G = (V, E)$ has a clique of size $k$. Formally we write this as:

$$\textsc{Clique} = \{\langle G, k\rangle : G \text{ has a clique size } k\}$$

**Theorem 9.1.** *The clique problem is* _____

Proof structure:

- We have already shown the problem _____
- We will show how to reduce every 3SAT problem to a clique problem.
- We will show that a YES instance of 3SAT is a YES instance of clique
- We will show that a YES instance of clique maps to a YES instance of 3SAT

**The reduction.** Consider an arbitrary instance of 3SAT, and let $k$ be the number of 3-clauses it contains, denoted by

$$C_1 \wedge C_2 \wedge \cdots \wedge C_k \tag{1}$$

For $r \in \{1, 2, \ldots k\}$, let the literals in the $r$th clause be denoted as $l_1^r, l_2^r, l_3^r$ so that:

$$C_r = (l_1^r \vee l_2^r \vee l_3^r) \tag{2}$$

In other words, $l_i^r$ is the $i$th literal in the $r$th clause.

For each literal $l_i^r$, we introduce a node $v_i^r$.

For each pair of literals $\{l_i^r, l_j^s\}$, we add an edge $(v_i^r, v_j^s)$ if:

- $r \neq s$
- $v_i^r$ is not the negation of $v_j^s$; this means they are *consistent*

**Let's consider an example** $(x_1 \vee \overline{x}_2 \vee \overline{x}_3) \wedge (\overline{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$

**The rest of the proof:** We need to prove that YES for the 3SAT instance maps to YES for the clique instance, and that NO for the 3SAT instance maps to NO for the clique instance.

This is equivalent to proving that the 3SAT instance is satisfiable if

It turns out this is true but we will not prove it. The outline of the proof is:

- ( $\Longrightarrow$ ) Assume there is a satisfying assignment, and prove that the literals set to TRUE to satisfy the $k$ clauses map to

- ($\Longleftarrow$) Assume that there is a $k$-clique in the reduced graph and prove that

## 10 The 2SAT Problem

Similar to 3SAT, the problem 2SAT is the task of finding a satisfying assignment for a boolean satisfiability formula in conjunctive normal form (an AND of ORs) when all clauses involve exactly two literals.

**Question 4.** *Consider the following two true facts.*

***Fact 1:*** *Every instance of 2SAT is an instance of SAT.*

***Fact 2:*** *You can reduce 2SAT to* Clique*, similar to the way we reduced 3SAT to* Clique

*Which of the following statements is true?*

A *Fact 1 implies that 2SAT is NP-hard, but Fact 2 does not*

B *Fact 2 implies that 2SAT is NP-hard, but Fact 1 does not*

C *Both facts imply that 2SAT is NP-hard*

D *Neither fact is sufficient to conclude that 2SAT is NP-hard*

## 11 Another reduction for 2SAT

Given an instance of 2SAT, construct the *implication graph* as follows:

- Create a node for each variable and its negation
- For a clause $(\ell_1 \vee \ell_2)$ (which may be positive or negative literals), add:
  - A directed edge $(\neg\ell_1, \ell_2)$
  - A directed edge $(\neg\ell_2, \ell_1)$

**An example.**

$$(x_1 \vee x_2) \wedge (x_1 \vee \overline{x}_3) \wedge (\overline{x}_2 \vee x_3) \wedge (x_4 \vee \overline{x}_3) \wedge (\overline{x}_1 \vee \overline{x}_3)$$

**Theorem 11.1.** *In the implication graph for the 2SAT instance, a variable and its negation will be in separate strongly connected components if and only if*